



Hochschule RheinMain
Fachbereich Design Informatik Medien
Studiengang Medieninformatik

Abschlussarbeit

zur Erlangung des akademischen Grades

Master of Science

L^AT_EX-Template für schöne Abschlussarbeiten

Vorgelegt von	Robin Kuschnereit
am	21. Juli 2025
Referent	Prof. Dr. Adrian Ulges
Korreferent	Viola Campos

Hinweis: Dieser Text ist wortwörtlich zu übernehmen, mit Ausnahme des grauen Abschnitts. Diesen legen Sie in Absprache mit dem Referent vor Beginn der Bearbeitungszeit fest.

Eigenständigkeitserklärung (gem. ABPO, Ziff. 4.1.5.4 bzw. RPO, §19.6)

Ich bin verantwortlich für die Qualität des Inhalts dieser Arbeit, den ich mit geeigneten wissenschaftlichen Quellen belegt bzw. gestützt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Texte, Gedankengänge, Konzepte, Grafiken, technischen Inhalte und ähnliches in meinen Ausführungen habe ich eindeutig gekennzeichnet und mit vollständigen Verweisen auf die jeweilige Quelle versehen. Alle weiteren Inhalte der Arbeit (Textteile, Abbildungen, Tabellen etc.) ohne entsprechende Verweise stammen im urheberrechtlichen Sinn von mir. Ich versichere außerdem, dass ich die Bachelor-Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Äußern Sie sich hier zur Verwendung generativer KIs. Dieser Abschnitt ist mit dem Referenten abzustimmen. Mögliche Formulierungen sind:

1. Verbot KI-generierter Inhalte: Ich versichere, dass ich die hier vorliegende Arbeit vollständig eigenständig erarbeitet, also keine KI-generierten Inhalte verwendet habe.
2. Erlaubnis KI-generierter Inhalte: Ich versichere, dass ich KI-Tools lediglich als Hilfsmittel verwendet habe und in der vorliegenden Arbeit mein gestalterischer Einfluss überwiegt. Ich verantworte die Übernahme jeglicher von mir verwendeter KI-generierter Inhalte vollumfänglich selbst.
 - (optional, falls Kennzeichnungspflicht) Unter „Übersicht verwendeter Hilfsmittel“ habe ich im Anhang der Arbeit sämtliche KI-generierten Inhalte einzeln aufgeführt und die jeweils verwendete KI-Anwendung benannt.
 - (optional, falls Prompts abzugeben) Die jeweiligen Benutzungseingaben (z.B. Prompts) sind der Arbeit beigegefügt.

Auch hiervon abweichende Regelungen sind in Absprache mit dem Referenten möglich.

Die vorliegende Arbeit wurde bisher weder im In- noch im Ausland in gleicher oder ähnlicher Form einer anderen Prüfungsbehörde vorgelegt. Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte als Täuschungsversuch gelten und prüfungsrechtliche Konsequenzen haben kann. Insbesondere kann es dazu führen, dass die Leistung nicht bestanden ist und dass bei mehrfachem oder schwerwiegendem Täuschungsversuch eine Exmatrikulation droht.

Erklärung zur Verwendung der Masterthesis

Hiermit erkläre ich mein Einverständnis mit den im folgenden aufgeführten Verbreitungsformen dieser Abschlussarbeit:

Verbreitungsform	Ja	Nein
Veröffentlichung des Titels der Arbeit im Internet	×	
Veröffentlichung der Arbeit im Internet		×

Wiesbaden, 21. Juli 2025

Robin Kuschnereit

Abstract

Contents

1	Introduction	3
2	Foundations	4
2.1	Large Language Models	4
2.2	Calibration	4
2.3	Metrics	4
2.3.1	ECE	4
2.3.2	Brier score	4
2.3.3	ASCE	4
2.3.4	GASCE	4
2.3.5	Skill score	4
2.4	Methods	4
3	Concept	5
3.1	Data generation	6
3.2	Evaluation through a LLM	10
3.3	Calibration	10
3.3.1	Histogram binning	12
3.3.2	Linear regression	12
3.3.3	Iterative group histogram binning	12
3.3.4	Iterative group linear binning	13
3.3.5	Regressor	14
4	Evaluation	16
4.1	Data	16
4.2	Models	16
4.3	Results	16

Contents	2
4.4 Experiments	16
5 Discussion	17
Sources	18
A Prompts	19
A.1 Evaluator LLM	19

Chapter 1

Introduction

Chapter 2

Foundations

2.1 Large Language Models

2.2 Calibration

2.3 Metrics

2.3.1 ECE

2.3.2 Brier score

2.3.3 ASCE

2.3.4 GASCE

2.3.5 Skill score

2.4 Methods

Chapter 3

Concept

Kurz motivieren: Was ist das Ziel? $P(\text{korrekt} \mid \text{LLM-generiertes Programm})$.

~~In the concept section we explain our experimental setup.~~ In the diagram 3.1 all main components that are used in our pipeline are shown. The process starts with the code generation prompt. This prompt is put into an LLM to generate a program. Besides the generated code we also fetch metadata like the token scores. We process this data to get a probability for each generated sample. After the generation, we evaluate the samples for correctness. This is done by using test cases. After the evaluation, we form groups which are determined by the sample features. Furthermore, we will explore an optional process which involves another LLM. This LLM is used to evaluate the generated code. The LLM evaluation output can either be treated directly as calibrated scores or be used for further calibration. After these step we use the out coming data for calibration. In the second phase we can choose between several calibration methods. These methods are histogram binning (HB), linear regression (LR), iterative group histogram binning (IGHB) or iterative group linear binning (IGLB). We measure the calibration on the data before the method is started and after the method was used. Besides using every method on their own its also possible to use every method at once. This is used to compare the outputs of each method.

The methods histogram binning, linear regression, iterative group histogram binning and iterative group linear binning are used from the paper multicalibration for confidence scoring in LLMs from Detommaso et al. [2]

This serves as a baseline.... to compare our calibrated probabilities against.

Klarstellen: Was sind diese Gruppen und wozu dienen sie?

Verstehe ich nicht. TBD.

Die Gruppen sind doch auch optional, oder?

Das ist ja keine zweite "Phase", sondern dein eigentlicher Betrachtungsgegenstand

Texte bitte
(deutlich)
größer

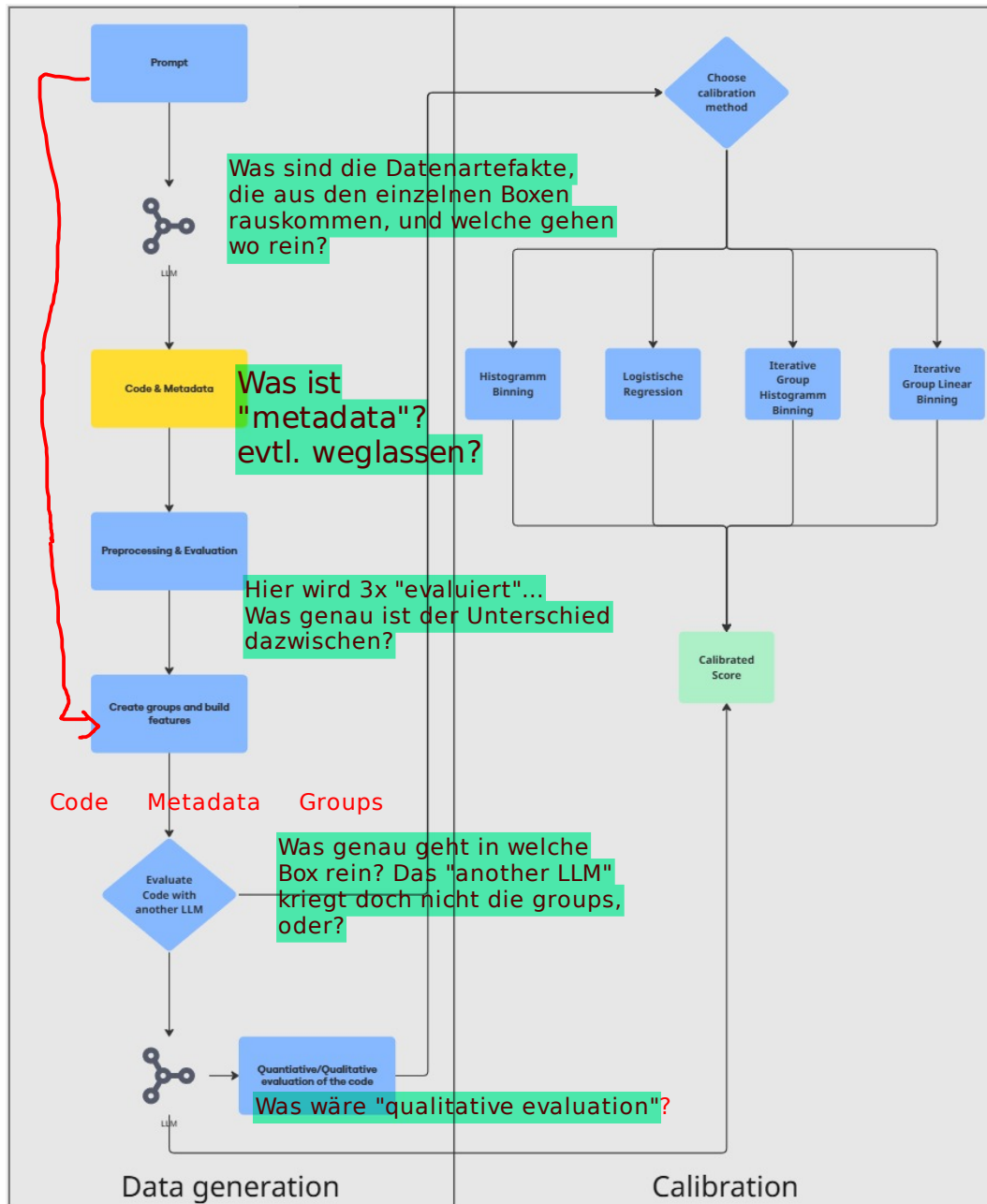


Figure 3.1: Calibration process flowchart

Ich würde erwarten, dass die Sections die Boxen in Figure 3.1 widerspiegeln...?

3.1 Data generation

Statt einer Section in mehrere unterteilen.

At the data generation beginning we got a prompt. This prompt specifies the criteria for the code to be generated. Depending on the dataset, this prompt consists of imports, a method ^{header??} hull and a doc string. In the method hull we find the function name, input parameters with their ^{nicht bei Python?} data type and the expected output data type. The doc string contains a description that specifies the functionality which is expected of the code. In most cases the prompt also contains examples. The examples specify cases

Running Example: Ein Komplett-Beispiel für die Datenerzeugung?

- Prompt
- Code
- Korrektheit
- Token Scores
- Gruppe

...

for which inputs which outputs are expected. In listing 3.1 we can see an example prompt from the humaneval dataset. This prompt is written in python.

Listing 3.1: Humaneval python task 19 example prompt [1]

```

1 from typing import List
2
3 def sort_numbers(numbers: str) -> str:
4     """ Input is a space-delimited string of numerals from 'zero' to '
5         nine'.
6         Valid choices are 'zero', 'one', 'two', 'three', 'four', 'five', 'six',
7         'seven', 'eight' and 'nine'.
8         Return the string with numbers sorted from smallest to largest
9     """
10    >>> sort_numbers('three one five')
11    'one three five'
12    """

```

The prompt is then forwarded to a LLM. The LLM then tokenizes and embeds the prompt. After this the LLM predicts the following tokens and chooses the token with the selected decoding strategy. and predicts the method body?

For the generation we use vllm [3]. Vllm is a fast and easy to use framework to make inference with LLMs. We use the LLM object to instantiate a model. The model can be selected through the model parameter. This can be any model from huggingface hub or a downloaded model. For the generation, we pass the prompts into the generate method. This method also takes sampling parameters. These specify parameters like the temperature, number of generations, stop words, max tokens and the count of probabilities. Which parameters we used during the experiment will be specified in the evaluation chapter 4.

Chapter 4 (konkrete chapters, figures, sections, equations werden groß geschrieben)

As response from the LLM, we get the generated token sequences. These sequences contain the completed code pieces. Besides these sequence we additionally fetch details about each token. This data contains an entry for each generated token. This entry contains the token id, the actual log probability, the rank and the actual decoded token. Furthermore, we fetch the cumulated token probability. Besides the information about the output tokens we also gather information about the input tokens. These data has the same form like the output data. Through that we can regenerate the probabilities for a given code generation.

In this work we used open source models. These model allow easy access to token probabilities and we can control what the influencing variables are. These token probabilities indicate how confident the model is in the selection of the token. Please note that the returned probabilities are log probabilities and need to be converted into normal probabilities to allow an intuitive assessment. In figure 3.2 we can see a correct generation for the prompt from listing 3.1. We can see that the generation was cut off at the first not intended comment. The colors indicate the confidence of the model in the token. A high confidence is colored dark green and a low confidence is colored in red. The colors where create with a continuous range from red to green where each percentage is represented by its own color.

Das könnte man auch erst bringen wenn's relevant ist.

Was ist das?

Was ist das?

Formaler aufschreiben:
- Erzeugte Tokens $t_1, \dots, t_n$
- Es sei $s_i = P(T_i = t_i | t_1, \dots, t_{i-1})$
der Score des i-ten Tokens...
- ...

Ist das überhaupt wichtig?

Doppelpunkt:

(Zeile x?)

z.B.?

unklar.

See highlighted code

Generated code:

```
# A dictionary to map numerals to their corresponding digits
num_map = {
    'zero': '0', 'one': '1', 'two': '2', 'three': '3', 'four': '4',
    'five': '5', 'six': '6', 'seven': '7', 'eight': '8', 'nine': '9'
}

# Split the input string into words and map each word to its corresponding digit
digits = [num_map[num] for num in numbers.split()]

# Sort the list of digits and map them back to numerals
sorted_digits = sorted(digits)
sorted_num_map = {num_map[num]: num for num in num_map}
sorted_numbers = ' '.join(sorted_num_map[digit] for digit in sorted_digits)

return sorted_numbers
```

Figure 3.2: Highlighted tokens with color that indicates confidence

The generation process is applied to every sample in the given dataset, ~~Through that we create one generation for each prompt. It would also be possible to create n samples for each prompt.~~ To summarize each sample has a prompt, the generated completed code and details about the token sequence. Furthermore we use stop words to cut of unwanted generations. These could be for example the keyword "class" in python. Because we assume that simple programs do not need a class for their solution. These stop words are described in more detail in the chapter 4.

After the generation process we check the correctness of the generated code completions. This is done by applying test cases. The test cases check if the generated code produces the correct output for a given set of input values. We mention at this point that test cases can have blind spots and can not cover every possible input-output combination. For the code to work we concatenate the prompt with the generated code completion and append the test cases. After this we got a program that we can execute. Depending on the result of the execution it is specified if the sample is correct or incorrect. If its executable and passes all test cases the sample gets the label correct. Fails the sample test cases or is not even executable it is labeled as incorrect.

To summarize we got at this point for every sample the program that consists of the prompt, code completion and the tests. The probability and the label which indicates if the sample is correct. For some calibration methods we discuss later we need features that describe the sample. We call these features groups. A group is a subset of the dataset. These groups can also be intersecting. For example a piece of generated code is written in the language python and has examples in the initial prompt.

The groups are associated with the samples through a matrix. This matrix has the shape number of samples and the number of groups. This matrix contains zeros and ones which display the affiliation of the sample and the groups. Each row is a sample and each column represents a group. A zero means that the sample does not belong to this group and a one means it does. This matrix is build during

1. Wie funktioniert das genau?
2. Dieser Punkt ist hier deplatziert. Du hast doch gerade per Summary das Thema "Generierung" abgeschlossen, wieso jetzt wieder ein technisches Detail?

Gruppen motivieren: Wir könnten z.B. erwarten, dass LLMs sich mit einer bestimmten Art von Programmen, z.B. besonders langen Programmen, oder Mathe-Problemen, schwerer tun. Für solche Gruppen von Problemen würden wir dann eine strukturell niedrigere Korrektheitswahrscheinlichkeit vorhersagen.

Formale Definition, am besten mit equation:
Label (...) = 1_ {executes and passes tests}

Der Satz gehört da nicht hin. *Erst* beschreiben was Du tust. *Dann* darfst Du es kurz diskutieren.

Themenwechsel -> neue section

Let N be the number of samples and G be the number of groups.

Then, the matrix is $N \times G$.

built

the data loading process and will be used in different calibration approaches in the next sections. A sample matrix is displayed in figure 3.3. The columns G_n stand for the groups and the row S_n stand for the samples. The set G contains all groups.

	G_1	G_2	...	G_n
S_1	0	1	...	1
S_2	1	0	...	0
	$\vdots$	$\vdots$	$\ddots$	$\vdots$
S_n	1	1	...	0

Nicht n Spalten und n Zeilen...

Figure 3.3: Sample group matrix

Ja, gibt der Matrix einen offiziellen Namen, damit man den später wieder verwenden kann!

In this work we use three different approaches to create groups. The first grouping style we introduce is named „simple“. This grouping style is our the entry point into the group topic. We use simple characteristics to build the groups matrix. The criteria we check for are: the prompt contains the word "example", is the prompt longer than the median prompt and is the program code longer than the median program code. ~~Reminder the program code consists of the prompt, code completion and tests.~~ The idea behind the example group is that it should intuitively be easier to create a program if you have examples that show how the code works. The same goes for the group that checks the prompt length. Here we we assume that the problem becomes more complex the longer the prompt is. For the last group, which checks the length of the generated program, we assume that longer generated code is more likely to be incorrect than shorter code.

Bullet Points?

Den Gruppen auch Namen geben?
"with example"
"long prompt"
"long program"
...

Our second grouping style approach is called „complexity“. For this we analyse the generated programs. As outputs of the analysis we get the lines of code, lines of comments and an approximation of cyclomatic complexity per program. ~~To get these statistics we need to convert our programs to actual files. That is because we use a third party tool for this analysis. This results are loaded and processed during our data loading process.~~ It should be noted that the complexity score we use is only an approximation because its difficult to automate the calculation over multiple different program languages because of their differences in programming paradigms [5]. We choose to create a group for different complexity levels. ~~This levels define the risk in software quality. They where defined by Thomas McCabe [6].~~

The first group contains all samples that have a complexity of 1-10. These samples have a small risk. The second group contains all samples that have a complexity of 11-20 and have moderate risk. The third group contains all samples with a complexity of 21-50 and have a high risk. The last group contains all samples that have >50 complexity and have a very high risk. These four groups contain all samples and are therefore self-contained.

The third grouping style places the samples into groups which represent „categories“. These categories are extracted from the task definition that is contained in the prompt. To achieve this we check the prompt for certain keywords and add them to the group. The first group is build to represent tasks that use mathematical operations and numbers. This assignment takes place through the check for keywords like 'integer', 'float' and a combination of 'number' and 'calculate'. The second group represents all samples that operate with strings or chars. Therefore we search for keywords like 'string', 'char' or 'word'. The last group contains all samples that use higher level data objects like lists. Here we check for keywords like 'list', 'array' or 'tree'.

OK

Wie??
Womit?

Das stimmt so nicht...
"Risiko" hat in der
Entscheidungstheorie
eine ziemlich klare
(andere) Definition.
Ist aber nicht wichtig
hier.

kurz erklären was das ist (und Referenz)

Welches?

cyclomatic?

are considered to

of being incorrect?

semantic categories of programming problems?

Ein bisschen motivieren 3.2 Evaluation through a LLM

One straightforward approach comes to mind to estimate our desired probability $P(\text{correct} | \text{generated code})$: It would be to simply ask another LLM if the generated code is correct. Such auto-evaluation is quite common in question answering, for example <https://autoevaluator.langchain.com/>

We also created a method to evaluate the generated code with the help of a LLM. The LLM can either be the same LLM or we can choose another one. This is helpful if we want to evaluate the code with a more powerful LLM. ^{Note that} The evaluator LLM does not need to be an open source model and it is also not necessary to retrieve the log probabilities. This is possible because we directly use the generated token sequences from the evaluator LLM and do not use the log probabilities. To get an evaluation of the generated programs, we build a prompt which contains the programming language, the program, which contains prompt, generated code and test cases, and an instruction to evaluate the correctness of the code. The actual prompt templates can be found in the appendix A.1. The prompts and approaches are inspired by the work of Ulmer et al. [4]. For the evaluation of the generated code we differentiate between a quantitative and a qualitative approach.

Erstmal beschreiben wie Du's machst. *Dann* kann man diese kleine Diskussion bringen.

Bullet Points?

The quantitative approach tries to directly get a ^{of correctness} percentage from the model. Therefore the model is asked to return a percentage from the range of 0 to 100. This percentage is then treated as the confidence in the correctness of the sample. We assume that ^{it is} it's hard for a model to rate the program by using a continuous number range.

The qualitative approach, on the other hand, gives the model the option to choose from a preset of qualitative descriptions. For example „High“ or „Very low“. These qualitative scale can be found in the Appendix A.1. Based on the selection the model makes the sample gets assigned a specific percentage. These values are defined independently and can also be adjusted after the model made its prediction.

Kleine Tabelle, welcher Wert auf welchen Prozentwert gemappt wird?

For both methods we use template matching to extract the values from the token sequences. For the qualitative approach we search for a number with a percentage symbol and for the qualitative approach we search for the predefined qualitative descriptions. ^{answer?} ^{use first occurrence?}

The probability outputs of these method can then be used for further calibration methods in the process. It is also possible to not use any calibration method and analyze the pure outputs from the evaluator LLM. :)

Das hier sind ja erstmal "Grundlagen von Kalibrierung"

3.3 Calibration

For our calibration approaches we need a probability which reflects the confidence in the generated code. ^{We obtain a first - baseline - estimate of this confidence from the log} To achieve this we are using the individual token probabilities. ^{generated} With them we calculate the average token probability per sample. This is done by accumulation of the token log probabilities. This number is then divided by the number of tokens. The result is then transformed into actual probabilities by using the exponential function. The formula is displayed in 3.1. We assume that these average probabilities are the confidence of the model in the generated code for further experiments. Lower average token probabilities imply ^{an} an uncertainty in the model output. Higher confidence scores imply that the model thinks that its output is reliable. As a results of this process ^{more} step we get an probability for each sample that represents the confidence.

We assume that - if the model is "insecure" about its generation - these token scores will be low / have a high entropy.

Besserer Name?

TBD

$$\text{probability} = \exp\left(\frac{\sum_{k=1}^n \log \text{prob}_k}{n}\right) \quad (3.1)$$

To review the calibration on our generated data, we need to split the probability range into bins. This is done by splitting the range into equally large bins. We control the number of bins with the parameter m . ^{Our average token probabilities are continuous numbers.} To assign this probabilities to a bin we need to know which probability comes into which bin. To achieve this we calculate the distance to

Section "Discretizing Scores"

Klarstellen:
Diese "rohe" Konfidenz dient für uns als Baseline. Die Schlüssel-Forschungsfrage ist, ob wir eine "besser" Schätzung mittels unserer Kalibrierungsmethoden gewinnen können.

each bin $\hat{p}$ choose the bin which is closest to the probability. The formula to calculate said assigned bin can be seen in 3.2.

Das ist eher als diskretisierte W'keit zu bezeichnen, oder?
Eine Art probability' ...

Definieren? $[1/m] = \{0, 1/m, 2/m, \dots, m-1/m, 1\}$

$$\text{assigned bin} = \operatorname{argmin}(|\text{probability} - \text{bin}|), \text{ where } \text{bin} \in \left[\frac{1}{m}\right] \quad (3.2)$$

With this assignment method we assign each sample a bin. This mapping is then used to calculate the correctness in each bin. This is done before and after the calibration method was applied. Through that we can evaluate the calibration on the uncalibrated data, the calibration on the calibrated data and the increase or decrease of metrics that were showed in chapter 2.

Das hier ist eher Evaluation. Würde ich bei die Gütemaße schieben.

Section
"Visualizing a Calibration Result"

calibration

To compare the approaches we calculate metrics on the uncalibrated data and then after each method was used. Through this we get an idea of how well each method performed under the same circumstances. Furthermore we use reliability charts and histograms to plot the changes that each method makes. The reliability charts show each bin and the correctness of the samples in said bin. The histograms show the distribution of the samples in the grid and can therefore track where the distribution mass moved. In figure 3.4 we can see an example for a reliability chart. On the x-Axis we got the confidence values that we received from the LLM. On the y-Axis we got the probability that a sample is correct for each bin. The blue diagonal line symbolizes the perfect calibration. The bar colors of the reliability diagram display the sample distribution. Darker blue shades mean more samples are in this bin and light blue or white displays less samples. Furthermore the numbers about the bars show the actual count of samples in this bin.

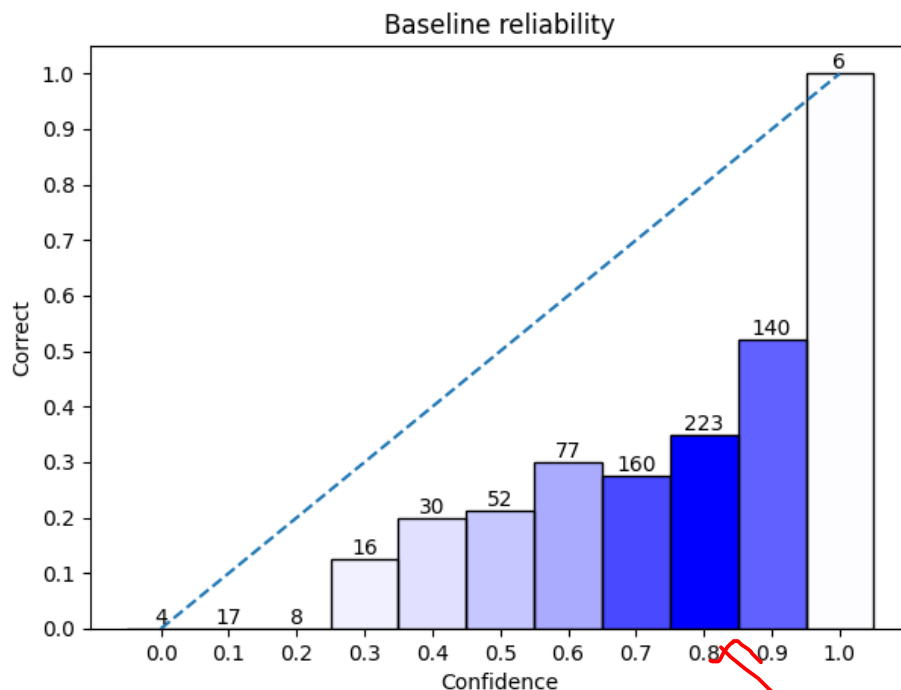


Figure 3.4: Reliability chart to display calibration

In figure 3.4 we can see the uncalibrated data distribution of a dataset. As we can see the probabilities are unevenly distributed and do not orientate them self on the diagonal line which would be the

are not close to

Pick Dir mal einen Balken raus und erkläre den:
- Hoher Anteil an Samples.
- Die rohen Scores drücken eigentlich eine hohe Konfidenz (ca. 80%) aus.
- Eigentlich sind aber nur ca 30% korrekt.
-> Schlecht kalibriert. Der Balken sollte 80% hoch sein.

perfect calibration of the underlying data. Perfectly calibrated data would mean that in bin 0.5 are always 50% correct and 50% incorrect samples.

Hier würde ich jetzt mit "3.2 Kalibrierung" anfangen.

... mit einer Definition: Eine Kalibrierungsmethode ist etwas, was folgende Eingabedaten kriegt.... und hieraus einen neuen Score erzeugt, von dem wir hoffen dass er eine bessere Schätzung der eigentlichen Korrektheit liefert.

3.3.1 Histogram binning

The first calibration approach is called histogram binning (HB). This approach is the simplest of all and tries to adjust all probabilities in a bin at once. To achieve the calibration improvement, we calculate the difference between the correctness and the average confidence in the bin for all bins. Because we discretized the values in each bin the average confidence is always the value of the bin. For example in the bin 0.5 the average confidence is 0.5. The correctness is calculated by using the label of the samples. With these two values we calculate the difference (delta) between optimal calibration and actual calibration. This is done for every bin. In formula 3.3 we can see the calculation of said deltas.

of those studied in this thesis.

Klarstellen: Deltas werden überacht gelernt, wir brauchen also eine Trainingsmenge

$$\text{delta}_b = \text{correctness}_b - \text{confidence}_b, \text{ where } b \in \left[\frac{1}{m}\right] \quad (3.3)$$

The calculated deltas are then used to update all the discretized probabilities in the bin by applying the delta values for the selected bin. To measure calibration again we need to repeat the process of bin assignment which was displayed in 3.2. The reason for this is that the deltas we calculated can result in values that are not exactly the discretized values of our grid. This process shifts the probabilities to a better calibrated version. But it neglects the fact that different bins and the contained samples are not always disjoint and do maybe depend on each other [2].

Das ist so leider unklar. Was heißt "applying"?

Evaluation -> gehört nicht hierhin.

3.3.2 Linear regression

The second method we use to calibrate data is linear regression (LR). In this approach the groups appear. The groups are used as input features for the linear regression. In formula 3.4 they are displayed as x_n . As output we want the difference between the label and the actual probability which is displayed as y . The label defines if a sample is correct or not and therefore can take the values zero or one. Which means incorrect sample will be shifted into the direction of zero because they become a negative sign and the correct sample will be shifted more to the direction of one because of the positive sign.

Das müsste (oben) mal rigide definiert werden: Was ist mit beidem gemeint?

Wie sind jetzt genau die x_i -Werte definiert?

$$y = b + w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n \quad (3.4)$$

To realize this the linear regression learns the weights w_n for each group and the bias b . In this approach we use the actual probabilities and not the discretized values. Furthermore the bias b of the linear regression is used to adjust samples that have no groups associated with.

Klarstellen: Muss überwacht gelernt werden. Wir benötigen eine Trainingsmenge.

With the learned weights $w_1, \dots, w_n$ and bias b we predict the adjustment for a given group feature input $x_1, \dots, x_n$. The returned delta values are then added to the actual probability of the samples. Through that we get a better calibrated probability for the sample. We also conducted a control experiment in which the actual labels are learned. The results will be shown in chapter 4.

Dann: Neue, finale Schätzung der W'keit = probability - y (oder so)?

3.3.3 Iterative group histogram binning

Iterative group histogram binning (IGHB) is a approach which combines the idea of groups with the concept of bins. Instead of calculating deltas for each bin, like we did it in the histogram binning 3.3.1

approach, we calculate these deltas for every bin-group combination. That is done by calculating the difference between correctness and the average confidence for every bin and the groups in the given bin as we see in formula 3.5. Through this we create a matrix of delta values which has is count of bins long and count of groups wide. Let X be the number of ..., and Y be the number of Then, ...

$$\text{delta}_{b,g} = \text{correctness}_{b,g} - \text{confidence}_{b,g}, \text{ where } b \in \left[\frac{1}{m}\right] \text{ and } g \in G \quad (3.5)$$

Problem: Sample kann zwar nur zu einem Bin gehören, aber zu mehreren Gruppen (-> mehrere Deltas).

But instead of applying the deltas to every sample in our data like we did with histogram binning, we select the combination of bin and group which has the most impact on the calibration. To calculate this we square our deltas and multiply them with the probability that a sample is in the bin-group combination. For the formula see 3.6 By doing this we get the bin and group which has the highest impact on the calibration of the data.

Das ist so kaum nachvollziehbar.

$$(b, g) = \text{argmax}(\text{delta}_{b,g}^2 * P(b \cap g)), \text{ where } b \in \left[\frac{1}{m}\right] \text{ and } g \in G \quad (3.6)$$

The adjustments of the sample probabilities are made in a loop. This loop stops if the maximal error in the data is smaller then a predefined value α . This α value also defines the grid size in this approach. The α value of 0.1 results in a grid of 10 bins. So the grid is defined by calculating $\frac{1}{\alpha}$ and using this value as the bin width. The max error on the other hand is recalculated on each iteration of the loop. It is defined as the result of multiplying the GASCE with the probability of which a sample is in a given group as we can see in formula 3.7.

$$\text{error} = \text{argmax}(\text{GASCE} * P(g)), \text{ where } g \in G \quad (3.7)$$

Ich dachte nur eine bin,group-Kombi wird geändert?

In each iteration we adjust the probabilities on all subsets of our data. For evaluation purposes we track the changes which each iteration made. After each iteration we fit the calibrator with the adjusted probabilities. That means we calculate each deltas again and check if the error is still greater than the α value. Is that not the case we stop the adjustment and the calibration process is finished.

Through the specific adjustment we make for the bin-group combinations there is a high risk of overfitting on the data. That is caused by the small amount of data that is adjusted in each iteration.

To apply the learned changes onto an new sample we need to keep track of every adjustment that is made per iteration. Therefore we need to save the bin-group combination that was adjusted and the delta which was applied. It is important to mention that these changes need to be applied in the exact same order as they were saved. That is because through the changes of time samples are moved into other bins and could be affected by the next change in another way.

3.3.4 Iterative group linear binning

Iterative group linear binning (IGLB) improves the previous IGHB approach. Therefore three adjustments are made. First we not only apply the corrections for exactly one bin-group combination but instead for a set of bins. This set is defined by the parameter τ . A τ of zero means we adjust all values that are smaller than or equal the selected bin. Analog a τ of one means we adjust all values that are greater than or equal the selected bin. The parameter τ specifies this cumulative distribution function. The update of a larger number of samples is assumed to help to limit the likelihood of overfitting.

to cumulative bins instead of individual ones. A bin is now all with $P(X \leq x_2)$, not $P(x_1 \leq X \leq x_2)$.

Secondly we apply so called linear scaling. This scaling is defined by the formula 3.8:

$$X' = \text{expit}(\alpha + \beta * \text{logit}(X)) \quad (3.8)$$

within each bin

risk

Das braucht eine konsistente Notation über alle Ansätze! TBD

X are the probabilities of the samples and X' are the new adjusted probabilities. To get an optimal set of alpha and beta values we minimize the MSE on given formula 3.8. This is done by using the Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm. As initial start parameter we use 0 for alpha and 1 for beta. By applying the formula we calculate the alpha and beta values for all tau-bin-group combinations. In the prediction phase we select the set which is needed for the adjustment.

We choose the subset we correct by calculating the probability that a sample is in a given subset with parameter tau, group and bin. We also do this analog with the delta values. This probability is then multiplied by the deltas squared. We get the tau-bin-group combination with the largest potential to improve calibration. This calculation is formalized in the formulas 3.9 and 3.10.

Hier wäre Pseudo-Code dazu sehr hilfreich.

$$\text{delta}_{\tau,b,g} = \text{correctness}_{\tau,b,g} - \text{confidence}_{\tau,b,g}, \text{ where } b \in \left[\frac{1}{m}\right], g \in G \text{ and } \tau \in [0,1] \quad (3.9)$$

$$(\tau, b, g) = \text{argmax}(\text{delta}_{\tau,b,g}^2 * P(\tau \cap b \cap g)), \text{ where } b \in \left[\frac{1}{m}\right], g \in G \text{ and } \tau \in [0,1] \quad (3.10)$$

The third improvement specifies conditions that lead to an early stopping of the algorithm. The early stopping is to prevent overfitting. The first early stopping criterion is the parameter ϵ . If ϵ is greater then the probability $P(\tau \cap b \cap g)$ of the chosen subset the algorithm will terminate. The second criterion checks if the MSE is still decreasing on the validation data set. If it stops decreasing or stays the same the algorithm will come to an halt.

Ist das willkürlich oder garantiert es irgendwas?

To apply the changes onto a new sample, that has not been learned, we need to apply all changes that where made during the iteration onto the sample. The changes need to be in order the where done while training.

Hier aufgehört zu lesen

3.3.5 Regressor

In the last approach we choose to use different regressors. With these regressors we predict the correction terms which are used to update the probabilities like in the approach 3.3.2. But with the modification that we use different input features for the regressor.

The new input features consists of the sample groups, the assigned sample probability and a histogram. The groups did not change and are used like we described them in section 3.1. For the probability we used the average token probability. The histogram was created by gathering the individual token log probabilities. These were transformed into normal probabilities and summarized using a histogram. The histogram bins were splitted into 10 bins. A sample histogram can be seen in figure 3.5. We assume that the distribution of token scores can correlate with the correctness of a sample.

Was sind diese Änderungen?
alphas? betas? deltas?
Sollten aus dem Pseudo-Code
(links) rauspurzeln.

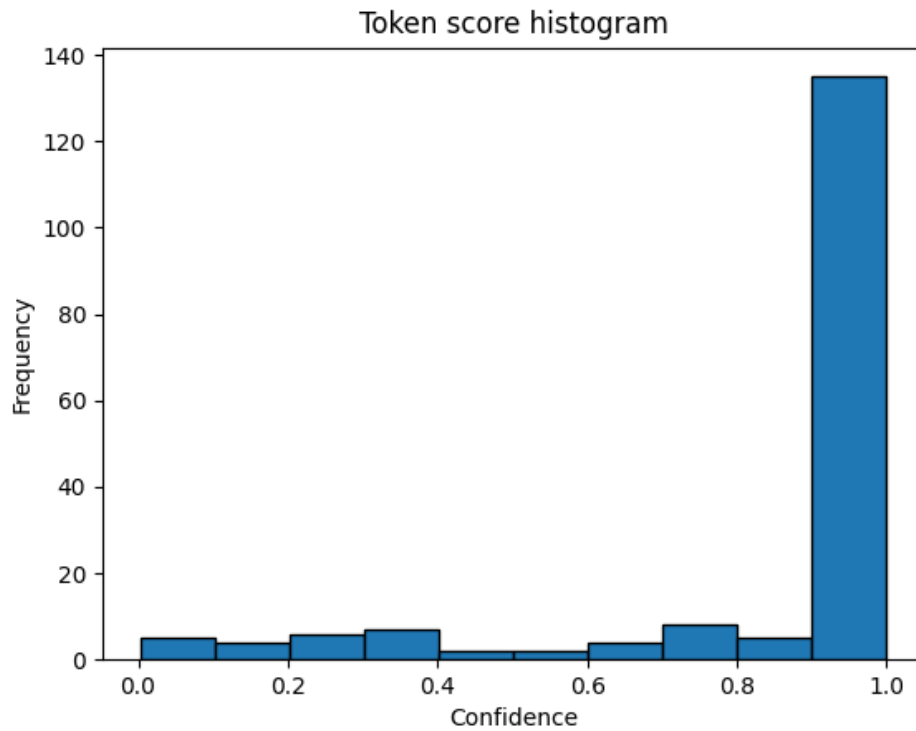


Figure 3.5: Histogram that displays the distribution of the token confidence in the humaneval task 19

This input features were used to train the regressor. As target we predict the deltas to adjust the probabilities for better calibration. As regressor we use linear regression, support vector regression (SVR) and XGBoost.

The linear regression is used like described in section 3.3.2, but with the modified input features.

For the SVR we use the radial basis function and for XGBoost we used the default settings to fit our data. All three regressors get the same input features and same target value. We input our features and learn the deltas for calibration. For the prediction part we apply the outputs of the regressor to the sample probability. We assume that the regressors learn deltas that improve the calibration of our data. At this point no bin assignments are made for the calibration process.

Chapter 4

Evaluation

4.1 Data

For this work we also investigate the generalization abilities of the different calibration approaches. Therefore we train the approaches on 60% of the underlying data. The other 40% are used for validation (20%) and testing (20%) of the learned parameters. We decided to use cross validation only in certain cases because the data is sufficiently large enough. The output of each method is stored in a folder structure. This folder structure is created while loading the for the calibration approach. It changes dynamically when the approach is called with another parameter set over the command line. The following is a example how the structure looks: „runs/humaneval-all-keep-Qwen2.5_Coder_7B-Instruct-1.0-comp-1/lr/linear/avg_logprob/categories/split“. The runs folder contains all results that are generated. The second folder is the run which describes the dataset and for code generation also the model with which the samples where generated. After this the calibration approach is specified in this case its „lr“ and under this folder we find the different binning types. For this work we only considered linear binning so its the only possible option. The next option specifies which method is used to generate the probability which is associated with the sample. The options are the average token probability (avg_logprob), the quantitative or the qualitative probabilities which where generated by the evaluation LLM. After this we got folder which specify the grouping style which were explained in the section 3.1. The last folder contains either the results for the partioned data or for the run with all data and no split into train, test and validation.

In the deepest folder we find charts which display the calibration before and after the method was used and a table which contains the calculated scores. Furthermore there is a file which shows the date at which time the data in this folder was generated.

4.2 Models

4.3 Results

4.4 Experiments

Chapter 5

Discussion

Sources

Literaturverzeichnis

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.
- [2] Gianluca Detommaso, Martin Bertran, Riccardo Fogliato, and Aaron Roth. Multicalibration for confidence scoring in llms, 2024.
- [3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the 29th Symposium on Operating Systems Principles, pages 611–626, 2023.
- [4] Dennis Ulmer, Martin Gubri, Hwaran Lee, Sangdoo Yun, and Seong Oh. Calibrating large language models using their generations only. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 15440–15459, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

Online-Quellen

- [5] Ben Boyter. Sloc cloc and code (scc). [letzter Zugriff: 6. Jun. 2025].
- [6] Thomas McCabe Jr. Software quality metrics to identify risk. [letzter Zugriff: 6. Jun. 2025].

Appendix A

Prompts

A.1 Evaluator LLM

"Answer as short as you can. Here is a snippet of generated code consisting of the prompt, generated code and test cases: {code}"

Verbalized quantitative prompt: "Please provide your confidence in the correctness of the provided code only in percent (0-100 %): "

Verbalized qualitative prompt: "Please provide your confidence in the correctness of the provided code only as one of {Qualitative scale:}; "

Qualitative scale: "
{
"Very low": 0,
"Low": 0.3,
"Somewhat low": 0.45,
"Medium": 0.5,
"Somewhat high": 0.65,
"High": 0.7,
"Very high": 1,
}"